

Presentation Defense Guide

Multivariate time-series anomaly detection — CNN autoencoder. Validation F1 $\approx$ 0.60, competition-portal F1 = 0.63.

How to use this: read Sections 1–4 to master the pipeline, Section 6 to rehearse answers, and the final cheat sheet in the last minutes before you present.

1. The pitch (memorise this)

30 seconds: "We detect anomalies in an 18-sensor industrial distillation process. There are no anomaly labels for training, so instead of a classifier we train a dilated-CNN autoencoder to reconstruct *normal* sensor windows; anything it reconstructs badly is flagged. We score with a Mahalanobis distance over per-sensor reconstruction error, threshold it on a small labeled validation set, and reach F1 = 0.63 on the competition portal."

2 minutes (add): the training data is unlabeled and not guaranteed anomaly-free, which shaped every choice. We rebuilt the code from notebooks into a tested package, fixed six correctness bugs, and improved the model over five rounds (checkpoint selection, a dilated TCN architecture for wide temporal context, Mahalanobis scoring, a precision-weighted threshold). We also rigorously tested fusing the CNN with two tabular detectors (Isolation Forest, then LOF) using leakage-free cross-validation — both were rejected, so we ship the CNN alone: a focused model we fully understand.

2. The pipeline, end to end (what / why / effect / alternative)

Stage	What it does	Why	Alternative & why not
Load per-run	Each of 28/10/53 runs kept separate	Runs are independent experiments	Concatenate all — would splice unrelated runs (a bug we fixed)
StandardScaler (train only)	Each sensor $\rightarrow$ mean 0, std 1	Stops high-variance sensors dominating MSE loss	Min-max — more outlier-sensitive; fitting on val/test leaks
Windowing 200 $\times$ 18, stride 10 (train) / 1 (eval)	Slice contiguous 200-step windows per run	Anomalies are slow drifts needing wide context	Point-wise/short windows — blind to drifts (notebook 04 showed F1 rises with window size)
CNN autoencoder	Compress $\rightarrow$ reconstruct normal windows	No labels $\rightarrow$ learn normal, flag deviations	Supervised classifier — impossible (no anomaly labels)
Training MSE + Adam, checkpoint sel.	50 epochs; keep best-validation epoch	Val score peaks ($\sim$ epoch 45) before train loss does	Take last epoch — ships an over-trained, worse model
Per-sensor error	MSE over time $\rightarrow$ one value per sensor	Keeps which sensor is off	Flat mean error — dilutes one sensor's spike with another's noise
Mahalanobis (Ledoit-Wolf)	$(x-\mu)^T \Sigma^{-1} (x-\mu)$ vs normal model	Weights each sensor by its own normal variability	Euclidean/flat MSE — ignores per-sensor scale & correlations
Per-timestep expand	Average overlapping window scores	Submission is per-timestep	Broadcast a block label — coarse, mislabels short anomalies
Threshold F-beta on val	Sweep PR curve, pick best cutoff	Turn a score into 0/1; $\beta=0.5$ curbs over-flagging	Fixed/quantile threshold — not tuned to the data
Evaluate	P/R/F1/ROC-AUC	AUC = ranking quality (the ceiling)	Accuracy — meaningless under class imbalance

3. Worked numeric example (real validation window)

A real 6-timestep slice from validation run 1 ($t = 789\text{--}794$), which is labeled anomalous. We follow two sensors: **T701** (a temperature) and **FT703** (product-stream flow — the strongest anomaly-signal sensor).

Step 1 — raw values, then standardize with the train statistics (T701: mean 370.4, std 58.4; FT703: mean 0.17, std 0.155). $z = (x - \text{mean}) / \text{std}$:

t	T701 raw	T701 z	FT703 raw	FT703 z	label
789	445.5	+1.29	0.1	−0.46	1
790	445.5	+1.29	0.1	−0.46	1
791	445.5	+1.29	0.1	−0.46	1
792	445.5	+1.29	0.2	+0.19	1
793	445.5	+1.29	1.1	+5.99	1
794	445.6	+1.29	0.3	+0.83	1

Note the signature: T701 sits flat and elevated ($z \approx +1.29$); FT703 **spikes to $z = +5.99$** at $t = 793$ — six standard deviations above normal flow.

Step 2 — the CNN reconstructs the window. Trained only on normal data, it 'expects' FT703 to stay near its usual range, so it reconstructs the spike poorly. Per-sensor squared error (illustrative, standardized units):

sensor	actual z (t=793)	CNN reconstructs	per-sensor error
T701	+1.29	+1.27 (easy, flat)	≈ 0.04 (small)
FT703	+5.99	+0.35 (expects normal)	≈ 4.8 (large)

Step 3 — Mahalanobis distance vs the normal error model. Suppose the normal per-sensor error has mean $\mu = [0.05, 0.20]$ and (diagonal, for illustration) covariance giving $\Sigma^{-1} = \text{diag}(1000, 50)$ — T701's error normally varies little, FT703's more. For our vector $x = [0.04, 4.8]$:

$\text{diff} = x - \mu = [-0.01, 4.60]$. $d^2 = 1000 \cdot (-0.01)^2 + 50 \cdot (4.60)^2 = 0.1 + 1058 = \mathbf{1058}$. For a normal vector $x = [0.06, 0.25]$: $d^2 = 0.1 + 0.13 = \mathbf{0.23}$.

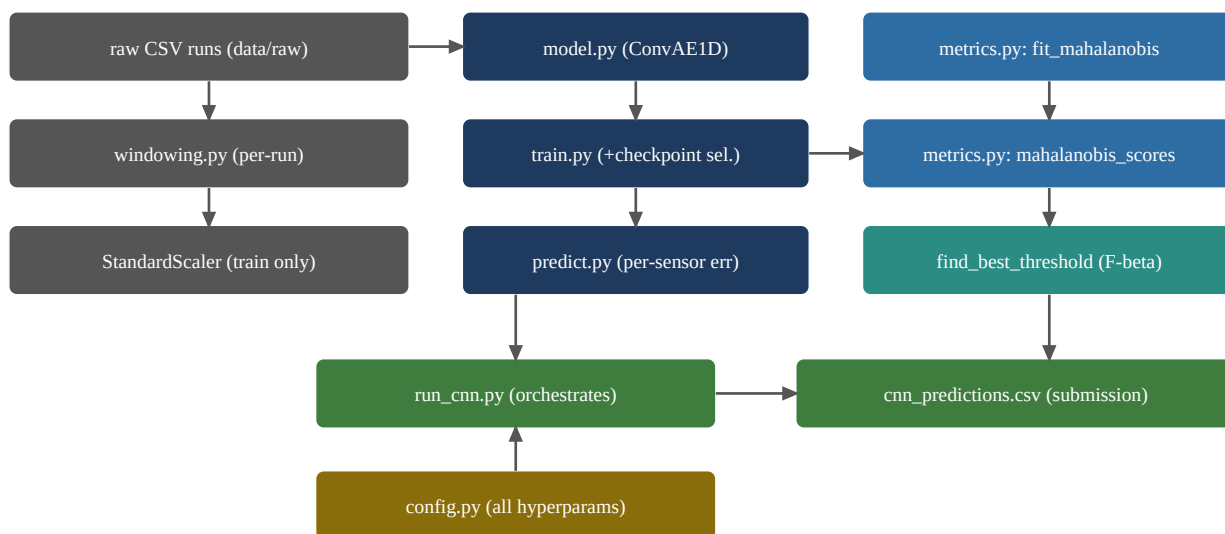
Step 4 — decision. The validation-selected threshold is ≈ 43 (raw Mahalanobis units). $1058 \gg 43 \rightarrow$ **flagged anomalous** (correct); $0.23 \ll 43 \rightarrow$ normal. The single FT703 spike drove the score, exactly because Mahalanobis weights it by FT703's own variability instead of averaging it away.

4. Concept primer (every method, one bite each)

- **Autoencoder** — a network that compresses input through a bottleneck and reconstructs it; trained on normal data, it reconstructs anomalies poorly, so reconstruction error is the anomaly score.
- **1-D convolution** — slides a small learned filter along time to detect local temporal shapes; sensors are the input 'channels'.
- **Dilated convolution** — spreads the filter's taps apart (gaps of 1,2,4,8) so stacking a few layers 'sees' ~ 63 timesteps of context with few parameters — needed for slow drifts.
- **Residual connection** — output = $F(x) + x$; lets gradients flow through deep stacks and lets each block learn a refinement (from ResNet/TCN).
- **Batch normalization** — rescales layer activations per mini-batch to stabilise and speed training.
- **Max-pooling / bottleneck** — downsamples time; the narrow middle forces compression so the AE can't just copy input to output (which would reconstruct anomalies too).
- **StandardScaler (z-score)** — $(x - \text{mean}) / \text{std}$ per feature, fit on training data only to avoid leakage.
- **Mahalanobis distance** — distance from a point to a distribution that accounts for each dimension's variance and the correlations between them: $(x - \mu)^T \Sigma^{-1} (x - \mu)$.

- **Ledoit-Wolf covariance** — a shrinkage estimator of Σ that stays well-conditioned (invertible) even with few samples relative to dimensions.
- **Precision / Recall / F1** — precision = fraction of flags that are correct; recall = fraction of real anomalies caught; F1 = their harmonic mean. **F-beta** with $\beta < 1$ weights precision more ($\beta = 0.5$ here).
- **Precision-Recall curve & ROC-AUC** — sweeping every threshold traces P vs R; ROC-AUC summarises the threshold-free ranking quality (our ceiling ≈ 0.76).
- **Isolation Forest** — ensemble of random trees; anomalies isolate in few splits (global, tree-based). Baseline; never earned fusion weight.
- **Local Outlier Factor (LOF)** — flags points whose local neighbourhood density is much lower than their neighbours' (local, density-based). Round-5 baseline; also rejected.
- **GroupKFold cross-validation** — k-fold CV where whole runs stay together in a fold, so no run leaks between train and test — used to choose the fusion weight honestly.

5. Component dependency map



Grey = data handling Navy = CNN model Blue = scoring Teal = thresholding Green = orchestration/output Gold = config feeds everything

Figure. config.py feeds all hyperparameters. run_cnn.py orchestrates: window & scale the data, train the CNN (with validation checkpoint selection), score per-sensor error, fit the Mahalanobis normal model, threshold on validation, and write the submission.

6. Likely questions & model answers

Q: Why an autoencoder and not a classifier?

A: There are no anomaly labels in the training data, so a supervised classifier can't be trained. An autoencoder learns 'normal' from unlabeled data and flags what it can't reconstruct.

Q: How is your anomaly score computed exactly?

A: Per-sensor reconstruction error (MSE over time, one value per sensor) turned into a Mahalanobis distance from a Gaussian model of normal error, so each sensor is weighted by its own normal variability and sensor correlations are accounted for.

Q: Why Mahalanobis instead of plain reconstruction error?

A: Flat error averages away which sensor is off — a genuine spike on a precise sensor gets buried under a noisy sensor's ordinary error. Mahalanobis rescales per sensor; it moved F1 from ~0.48 to ~0.60.

Q: Your training data may contain anomalies — isn't that a problem?

A: Yes, and we acknowledge it: the loss keeps falling while validation peaks around epoch 45, the signature of the AE slowly learning to reconstruct anomalies too. We mitigate with best-checkpoint selection, and the top future-work item is contamination-robust training (down-weighting likely-anomalous windows).

Q: How do you choose the threshold, and isn't that overfitting to validation?

A: We sweep the precision-recall curve on the 10-run validation set and pick the F-beta-optimal cutoff. It is a risk — everything is tuned on those 10 runs — but the portal F1 (0.63) exceeds validation (0.60), so we are not badly over-fit. A run-grouped CV (already built) would give a fully honest estimate.

Q: Why F-beta = 0.5 and not F1?

A: The score distributions overlap heavily, so plain F1 kept choosing a trigger-happy threshold flagging 65–88% of timesteps. $\beta=0.5$ weights precision to curb that. (Diagnostic caveat: if the portal grades plain F1, an F1-tuned threshold scores ~0.045 higher — a change we are testing.)

Q: What limits your F1 — why not 0.9?

A: ROC-AUC ≈ 0.76 : the score ranking genuinely overlaps, so no threshold separates cleanly. F1 is bounded by score separability, which is a model/score problem, not a threshold problem.

Q: Why dilated convolutions?

A: The anomalies are slow drifts. A plain kernel-3 conv sees only 3 timesteps; stacking dilations 1-2-4-8 gives a ~ 63 -step receptive field with no extra parameters, so the model can actually see the drift.

Q: Why did you drop the hybrid / Isolation Forest?

A: Across three fusion attempts (IF, raw-LOF, standardized-LOF) with leakage-free run-grouped CV, the tabular detector always got ~ 0 weight and never beat the CNN alone — its AUC (~ 0.67) is well below the CNN's (~ 0.77). Shipping the CNN alone is the honest, stronger choice.

Q: How do you prevent data leakage?

A: The scaler is fit on train only; windows never cross run boundaries; the Mahalanobis model uses only confirmed-normal windows; and fusion weights are chosen with GroupKFold so no run appears in both train and test folds.

Q: Why per-run windowing?

A: Each run is an independent experiment; concatenating them before windowing would create windows splicing two unrelated runs — a fake transient. We window each run separately (a bug we found and fixed).

Q: Is your validation estimate trustworthy?

A: Reasonably: the portal F1 (0.63) is slightly above validation (0.60), so validation is if anything pessimistic. The weakness is that only 10 labeled runs drive every tuning decision.

7. Limitations & future work (say these before they ask)

- **Score-separability ceiling** (AUC ≈ 0.76) — the main limit; needs a better score, not a better cutoff.
- **Training-set contamination** — 'normal' data isn't guaranteed clean; contamination-robust training (kNN-density loss weighting, CLEANet/RiAD-style) is the highest-leverage next step.
- **Single global threshold across runs** — diagnostics show raw scores vary $\sim 100\times$ per run, so two validation runs score F1=0 despite correct within-run ranking; per-run score normalization is being tested.
- **Everything tuned on 10 validation runs** — use the run-grouped CV for an unbiased estimate.
- **Metric mismatch** — we optimise F0.5 while the portal may grade F1; aligning them is $\sim$ free F1.
- **Run-to-run variance** — identical configs vary ± 0.02 F1; seed-ensembling would stabilise and lift it.

8. Cheat sheet (last-minute review)

One-sentence project: an unsupervised dilated-CNN autoencoder that learns normal 18-sensor behaviour and flags deviations via Mahalanobis distance; portal F1 = 0.63.

Pipeline in 9 words: load → scale → window → reconstruct → per-sensor error → Mahalanobis → expand → threshold → predict.

Key numbers: 18 sensors · window 200, stride 10/1 · 50 epochs, best ≈ epoch 45 · encoder dilations 1-2-4-8 (RF ≈ 63) · bottleneck 64×100 · $\beta = 0.5$ · val F1 ≈ 0.60, ROC-AUC ≈ 0.76, portal F1 = 0.63.

Three 'why' anchors: (1) no labels → autoencoder; (2) slow drifts → dilated + window 200; (3) one sensor matters → per-sensor Mahalanobis, not flat error.

The ceiling line: "F1 is capped by score separability (AUC 0.76); threshold tuning can't beat overlap — only a better score can."

The hybrid line: "We tested fusion three ways with leakage-free CV; the tabular detector always got ~0 weight, so CNN-alone is the honest answer."

If stuck, retreat to: "We learn normal, measure surprise per sensor, and pick a threshold on validation — and we validated every claim honestly, including the negatives."

Biggest weakness to own first: training data may contain anomalies (contamination); next step is robust training. Say it before they do.